

Block Diagram Report (3–4 pages)

MNEMOS 32: Multi-Core RV32IMA+P RISC-V SoC

Group Number: 22

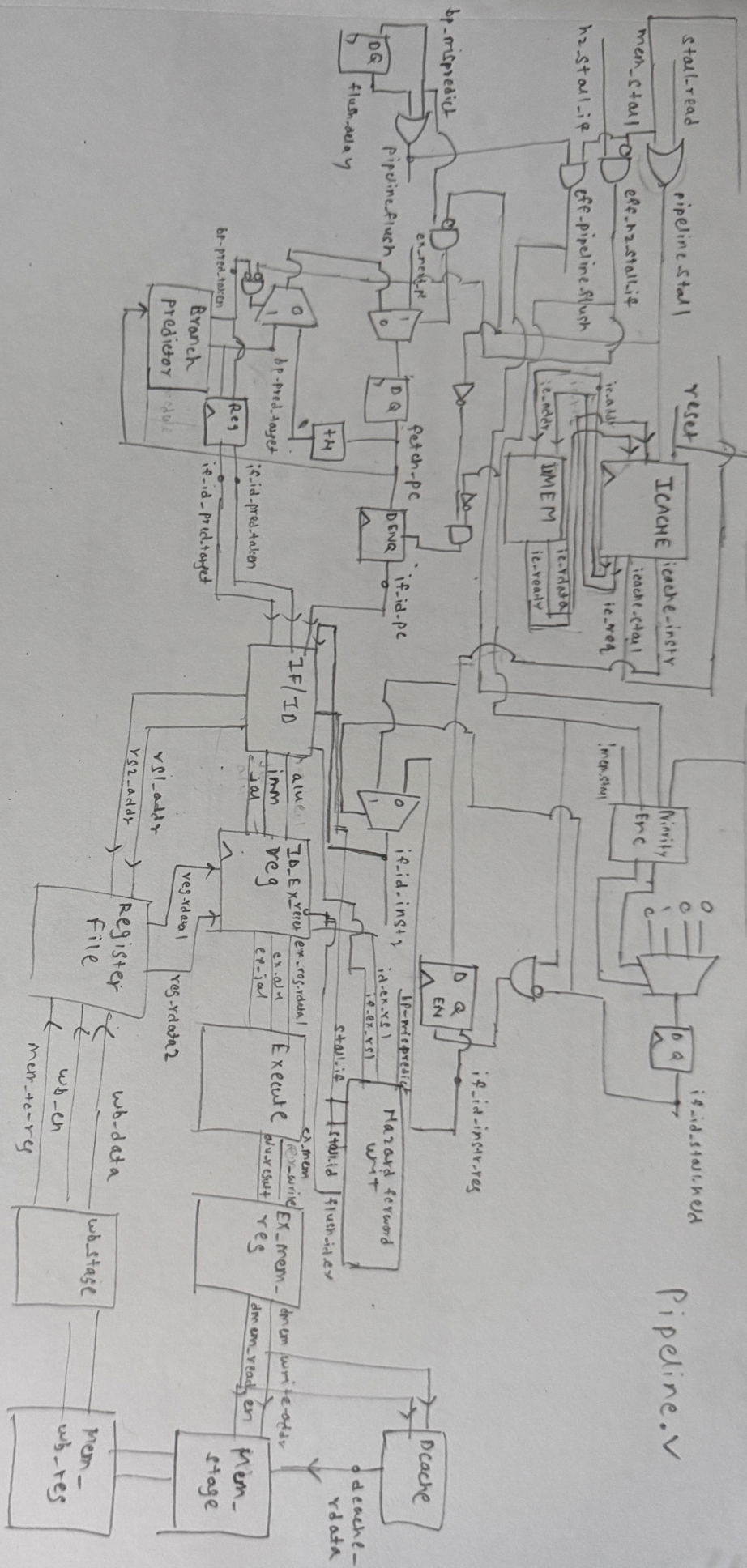
1. Top-Level Hardware Block Diagram (FULL PAGE)

Diagram must be clean, readable, and properly labeled

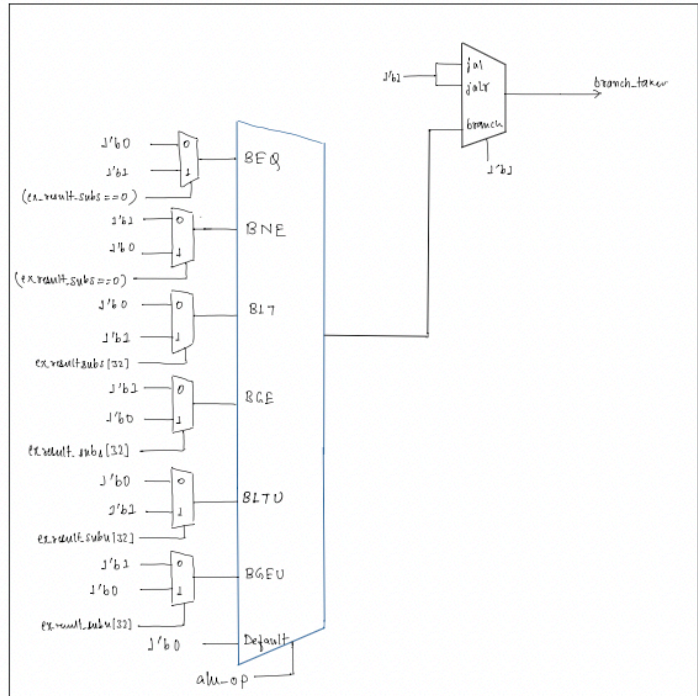
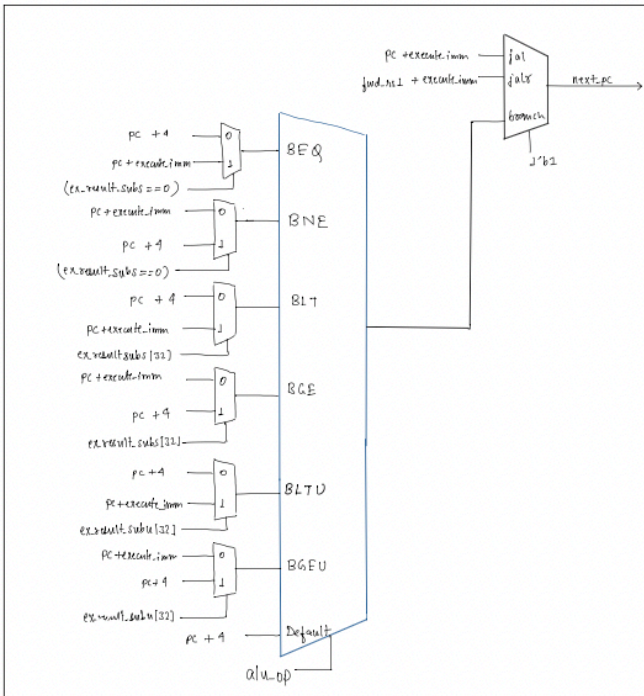
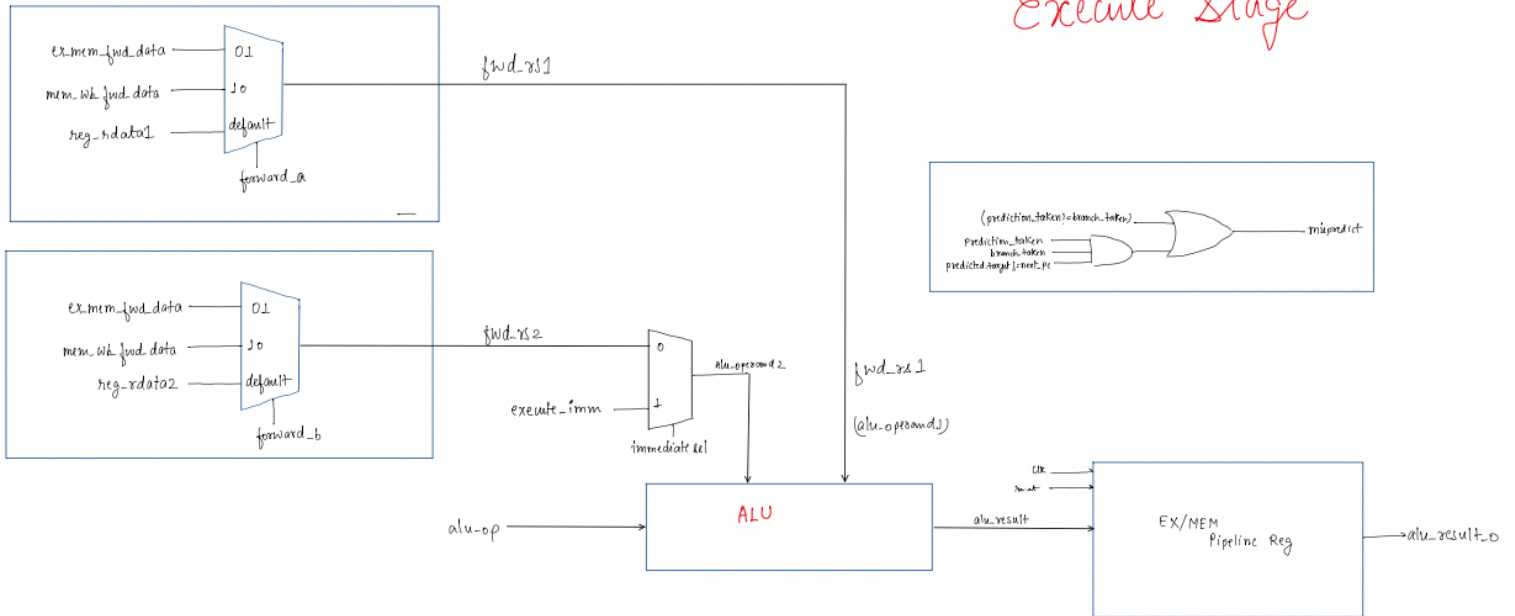
Must clearly show:

- Pipeline stages (IF, ID, EX, MEM, WB)
- All major modules (ALU, MUL/DIV, Control, Memory)
- Signal names + widths
- Pipeline registers (IF/ID, ID/EX, etc.)
- Control signals (at least key ones: RegWrite, MemRead, etc.)
- Clock and reset
- **New modules highlighted clearly**

Top Level Pipeline

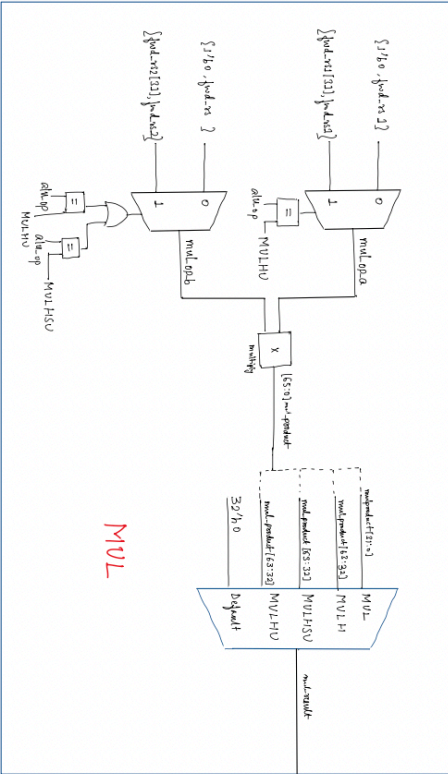


Execute Stage

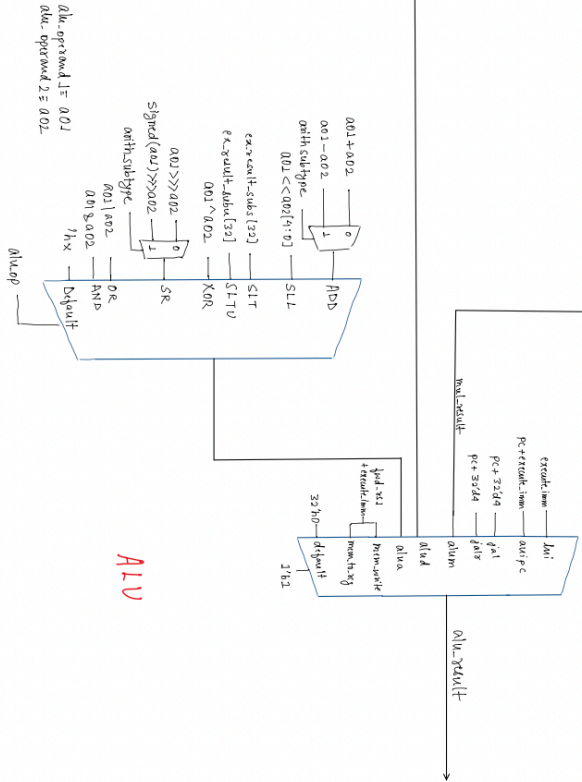
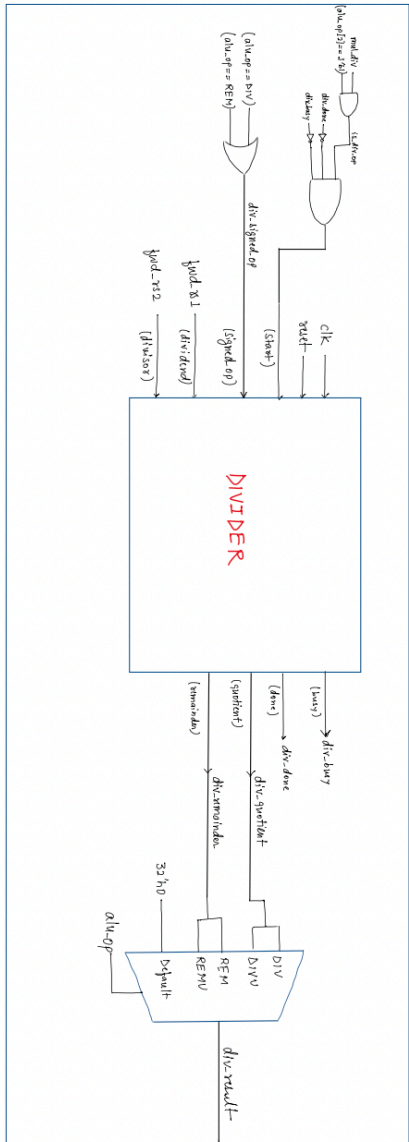


$$[32:0] \text{ ex_result_subs} = \{ \text{alu_operand1}[31], \text{alu_operand1} \} - \{ \text{alu_operand2}[31], \text{alu_operand2} \}$$

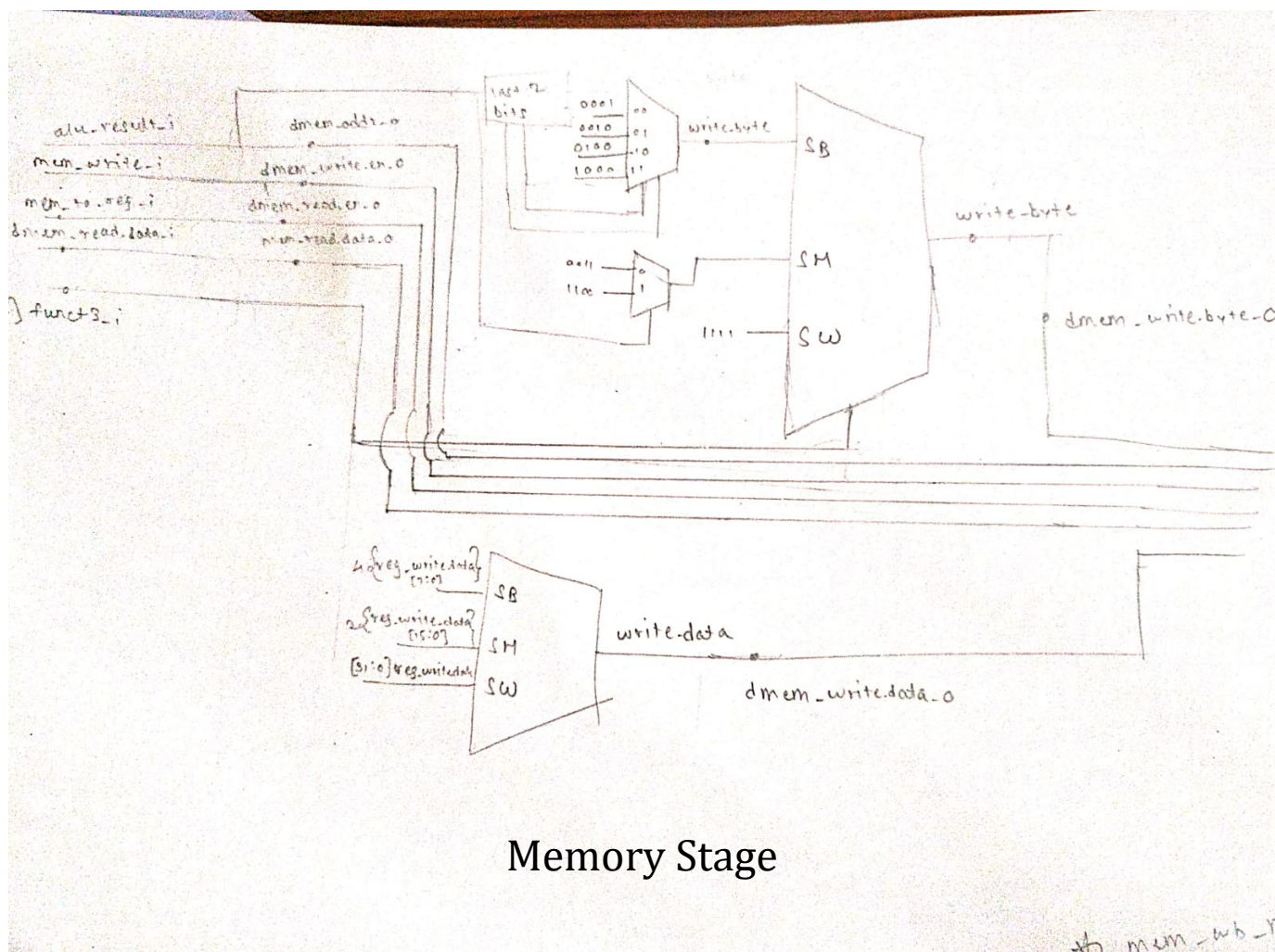
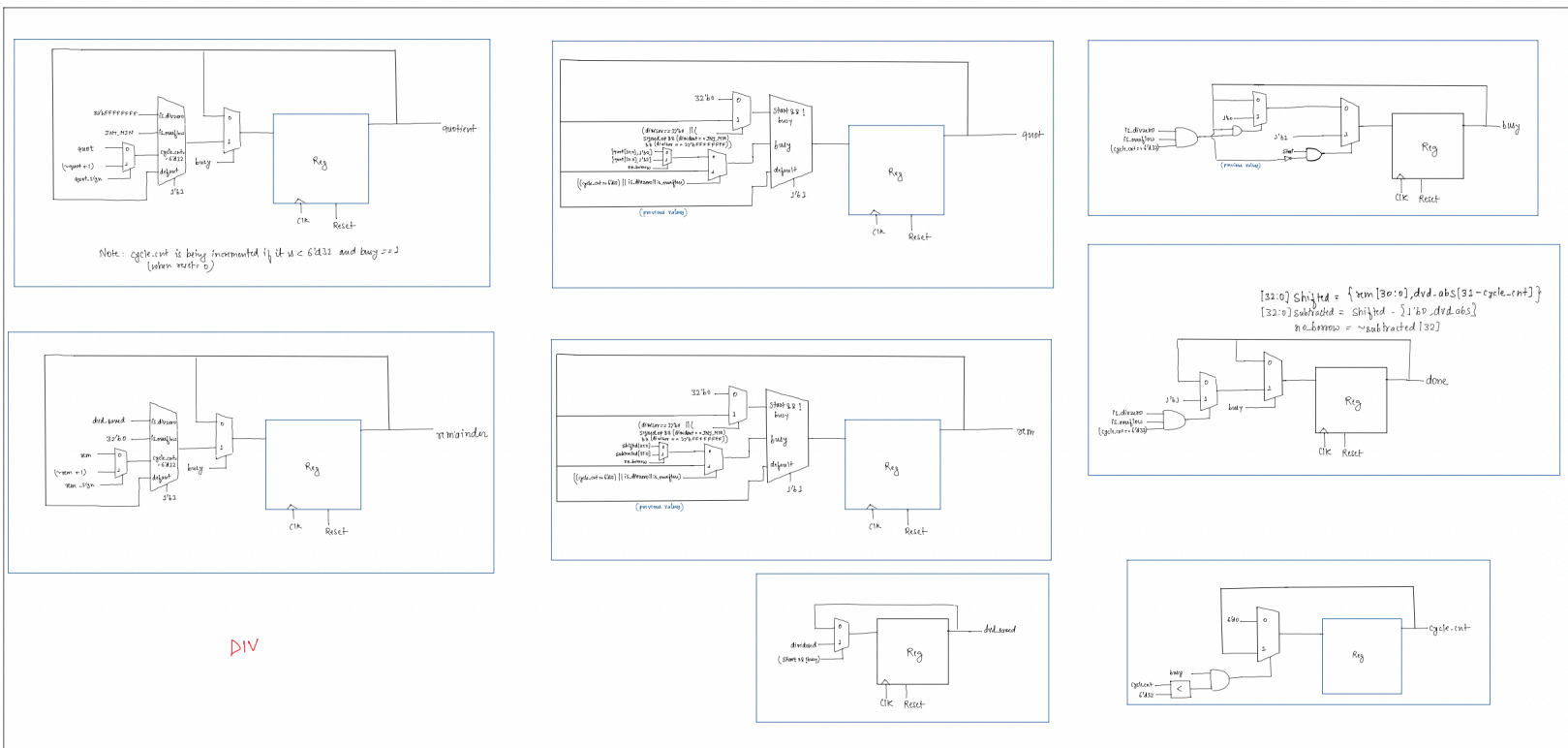
$$[32:0] \text{ ex_result_subs} = \text{alu_operand1} - \text{alu_operand2}$$



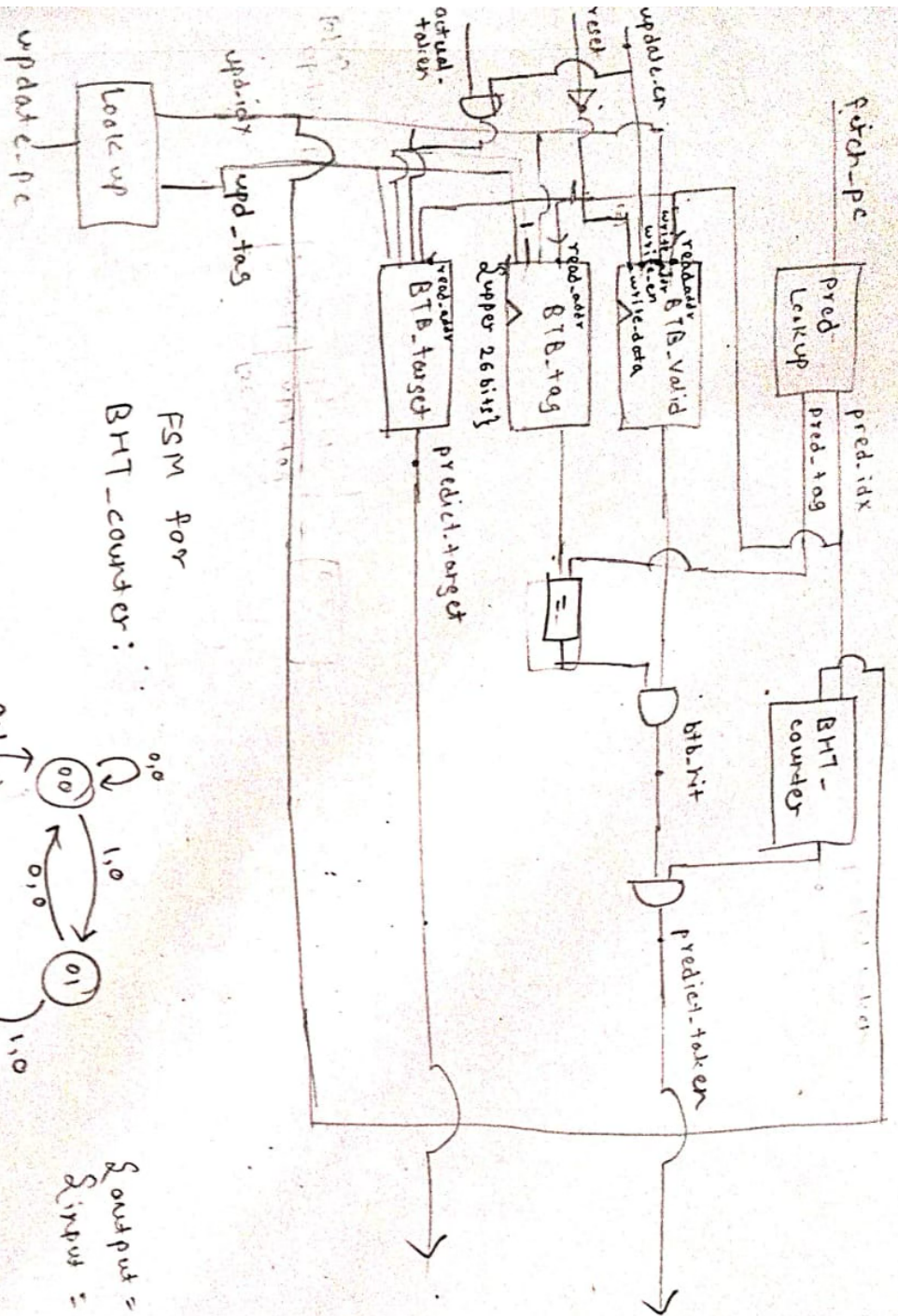
$$\begin{aligned} \text{div_m} &= (\text{div_s} \ll \text{mul_div_s} \ll \text{is_div_op}) \\ \text{div_d} &= (\text{div_s} \ll \text{mul_div_s} \ll \text{is_div_op}) \\ \text{div_a} &= (\text{div_s} \ll \text{mul_div_s}) \end{aligned}$$



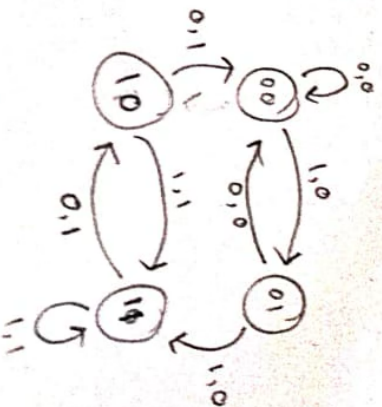
ALU



Branch Predictor



FSM for
BHT-counter:

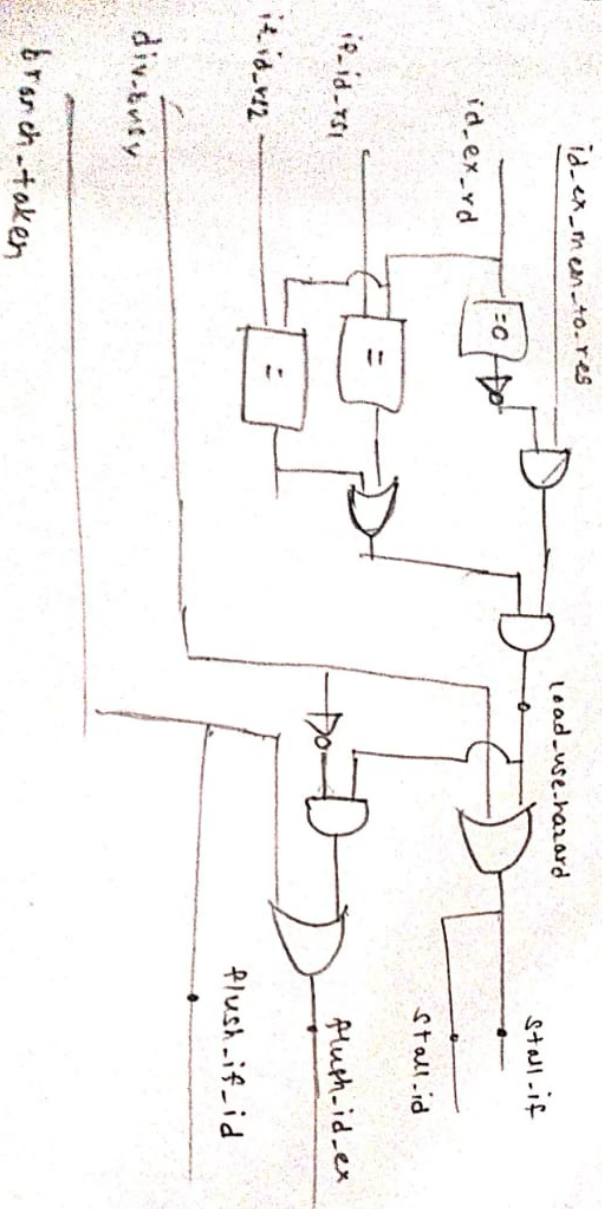
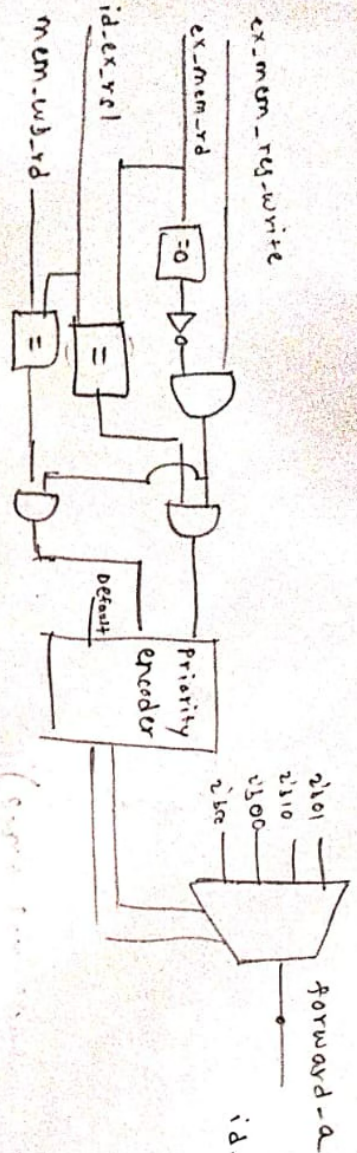


Output = predict-token
Input = actual-token

In code it's constructed
using if/else.

Hard Forward Unit

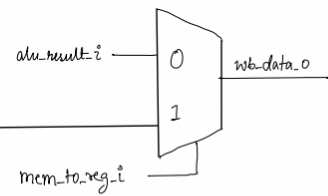
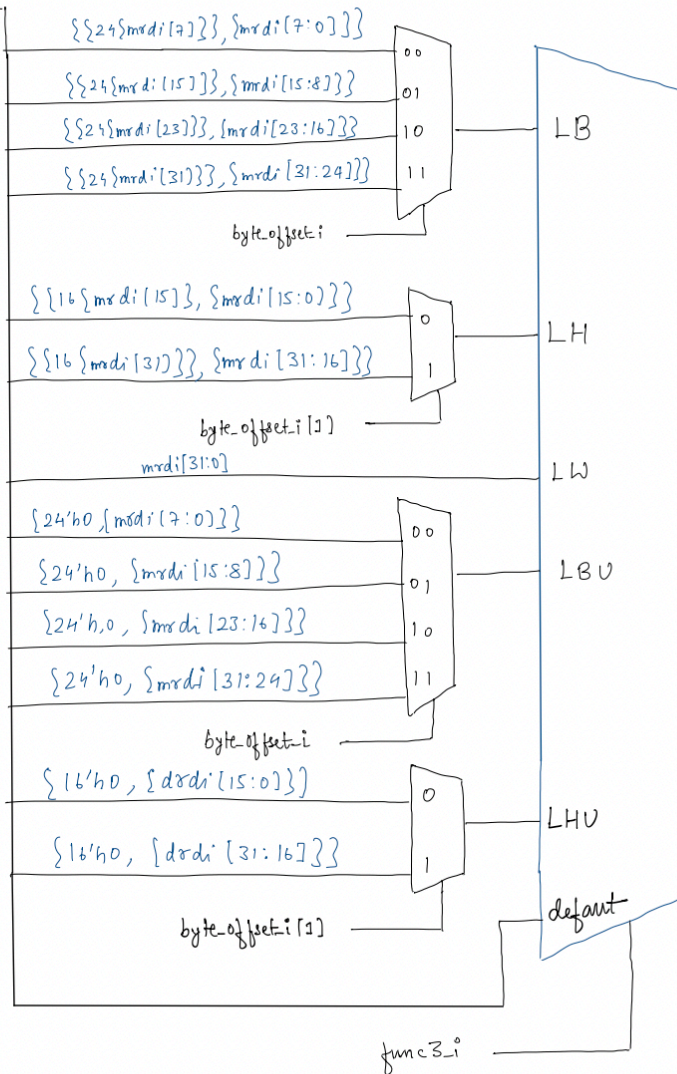
Same for forward-b for
id-ex-rs2,



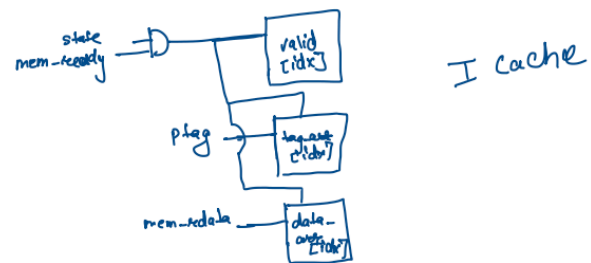
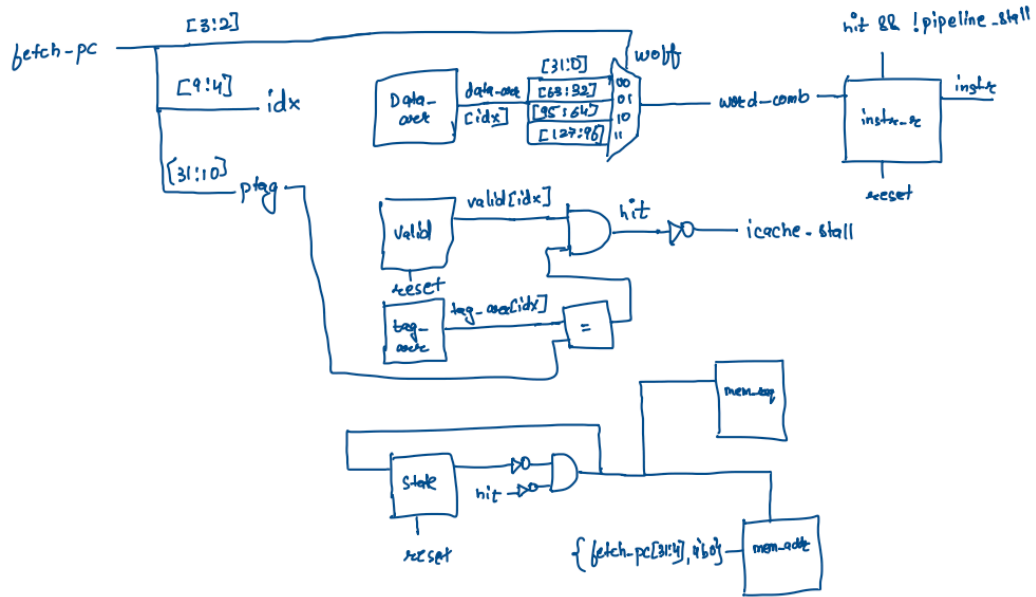
WB

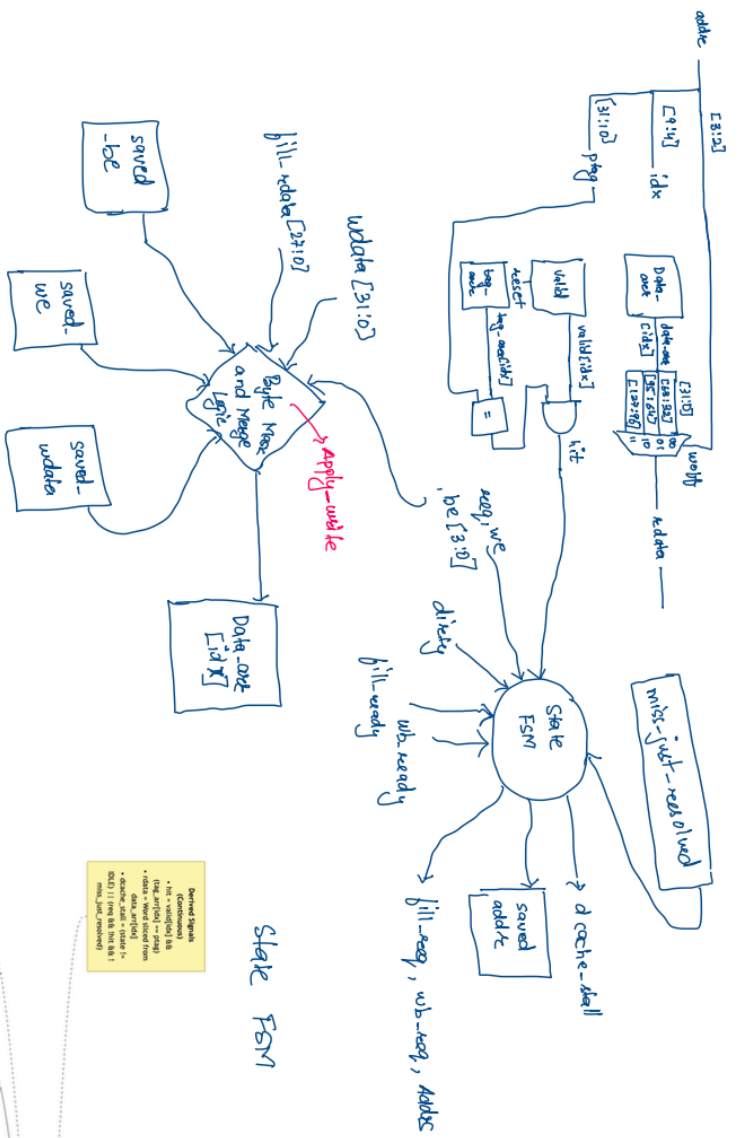
mem-
read_data-
i

mrdata = mem-read_data-i

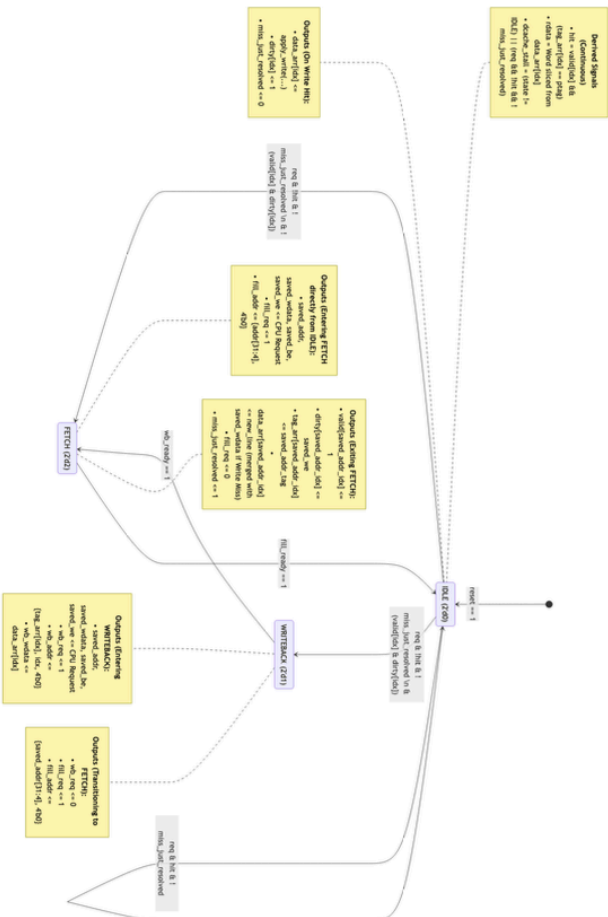


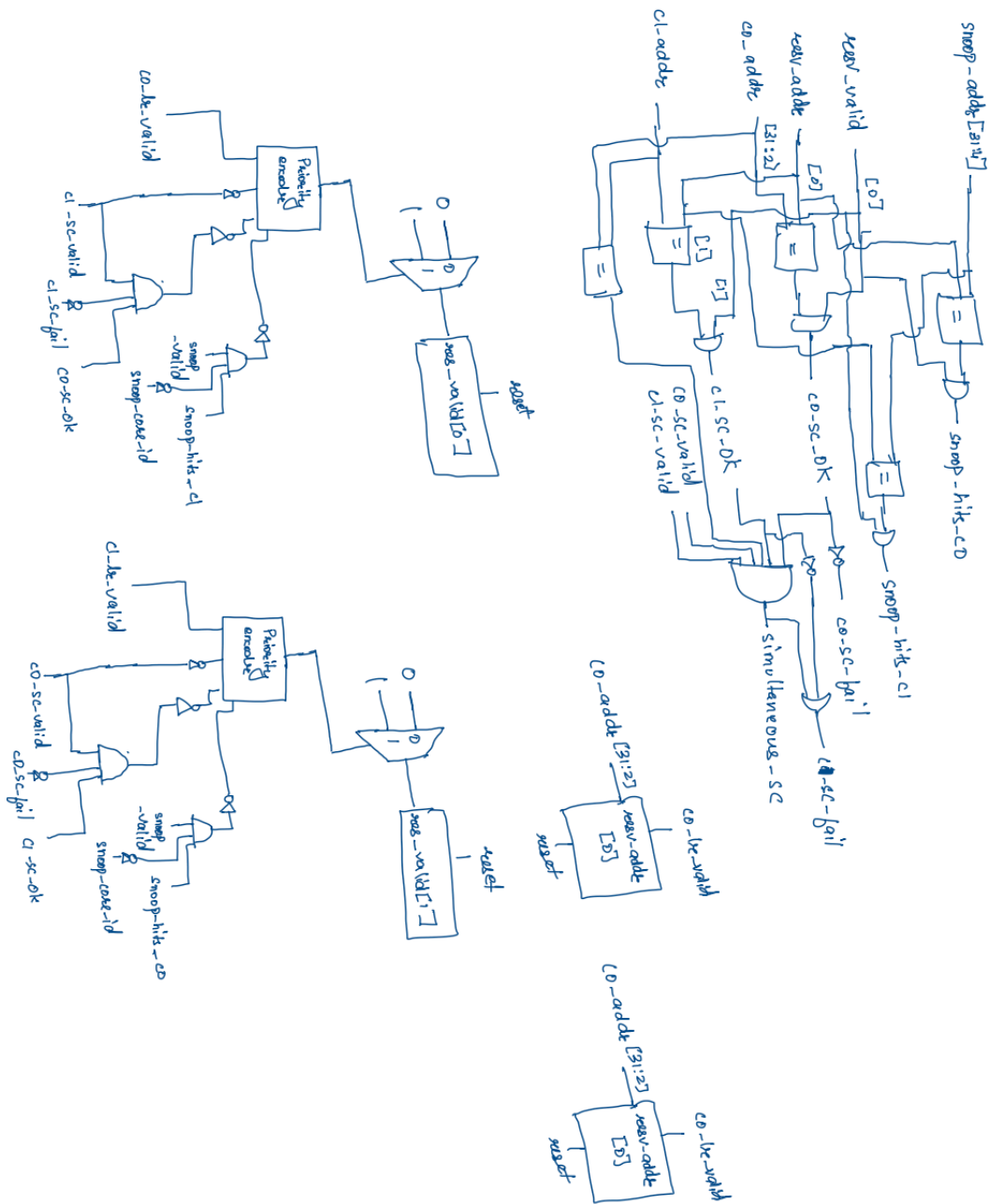
ICache





State FSM





Exclusive Monitor

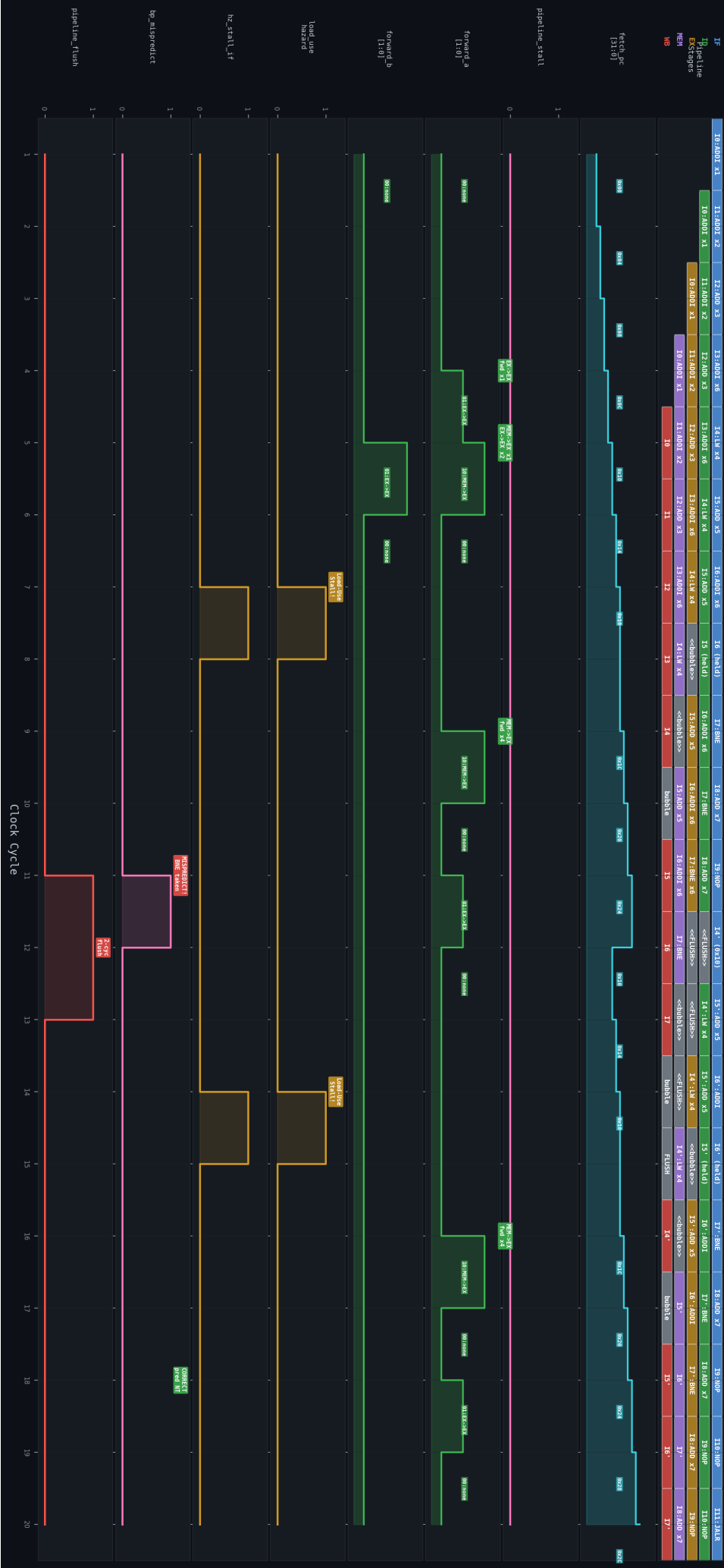
2. Timing Diagram (FULL PAGE)

Requirements:

- ≥ 20 cycles
- Must correspond to a **real instruction sequence**
- Must clearly show:
 - pipeline stages per cycle
 - MUL/DIV execution (multi-cycle if applicable)
 - stalls / forwarding / hazards

Idealized diagrams without hazards → not allowed

Code A: Hazard & Branch Stress Test (always-hit) | 5-Stage RISC-V | Cycles 1-20



3. Module Summary (Table Only)

- List all modules
- If the hardware contribution is currently limited (few modifications or modules), consider moving more functionality into hardware or adding new hardware features.

Module	Purpose	File	Software/Hardware module?
IF/ID Register	Pipeline register between Instruction Fetch and Decode stages	IF_ID.v	Hardware (Verilog)
Branch Predictor	Predicts branch outcomes to minimize control hazards	branch_predictor.v	Hardware (Verilog)
Bus Arbiter	Manages access to a shared memory bus between multiple cores/components	bus_arbiter.v	Hardware (Verilog)
Data Cache	Caches frequently accessed data to reduce memory latency	dcache.v	Hardware(Verilog)
Hardware Divider	Performs division operations (part of the 'M' extension)	divider.v	Hardware(Verilog)
Dual Core Top	Top-level wrapper for the dual-core processor system	dual_core_top.v	Hardware(Verilog)
EX/MEM Register	Pipeline register between Execute and Memory stages	ex_mem_reg.v	Hardware(Verilog)
Exclusive Monitor	Tracks load-reserved/store-conditional (LR/SC) for atomic operations ('A' extension)	exclusive_monitor.v	Hardware(Verilog)
Execute Stage	Contains the ALU and handles instruction execution	execute.v	Hardware (Verilog)

Hazard & Forwarding Unit	Detects data/control hazards and forwards data or stalls the pipeline	hazard_forward_unit.v	Hardware (Verilog)
Instruction Cache	Caches frequently accessed instructions	icache.v	Hardware (Verilog)
Memory Stage	Handles logic for loading from and storing to memory	mem_stage.v	Hardware (Verilog)
MEM/WB Register	Pipeline register between Memory and Write-Back stages	mem_wb_reg.v	Hardware (Verilog)
Main Memory	Top-level memory module or memory controller interface	memory.v	Hardware (Verilog)
Opcode Definitions	Header file containing constant definitions for instruction opcodes	opcode.vh	Hardware Header(Verilog)
Pipeline Top	Connects and manages the 5 stages of a single core pipeline	pipeline.v	Hardware (Verilog)
RV32IMA Core	Top-level wrapper for a single RISC-V core with I, M, and A extensions	rv32ima_core.v	Hardware (Verilog)
Dual Core Testbench	Simulation environment to verify the dual-core design	tb_dual_core.v	Simulation/Testbench (Verilog)
Pipeline Testbench	Simulation environment to verify the single pipeline core	tb_pipeline.v	Simulation/Testbench (Verilog)
FPGA Top	Top-level module customized for physical FPGA synthesis and board I/O	top_fpga.v	Hardware (Verilog)

Write-Back Stage	Logic for writing execution results or memory data back to the register file	wb.v	Hardware (Verilog)
------------------	--	------	--------------------

4. Hardware Resource Estimates

- Given the block diagram and hardware modules, estimate the usage of:
(initial estimates: <https://chatgpt.com/share/69c8b008-4f60-8320-a9d0-58af100f4fc1>)

Resource	Maximum Available	Estimated Usage	% utilization
BRAM	4860 Kbits	~1000Kbits	~20%
DSP blocks	240	~16	~7%
LUTs	63400	~15000-18000	~25%-28%
FF	126800	~10000-12000	~8%-10%

5. Verification Plans ($\leq$ 6–8 lines total OR 3–4 bullets)

Examples:

- How will MUL/DIV be tested?
- How will correctness be verified (edge cases)?
- How will processor-level execution be validated?

ALU Functional Testing: Validate MUL/DIV and arithmetic operations using a testbench and golden outputs.

Pipeline Hazard Validation: Stress-test the control and data forwarding logic by injecting instruction streams that create back-to-back Read-After-Write (RAW) data hazards along with branch instructions causing structural hazards

ISA Testing of RV32A: Run standard ISA test suites (such as riscv-tests) to ensure

End-to-End System Simulation: Execute compiled assembly or C benchmarks on the full RTL simulation

6. Gantt Chart

- $\geq 15+$ activities. Each activity must be concrete and testable
 - Implement Booth multiplier + testbench
 - Integrate MUL into EX stage
 - Add stall logic for multi-cycle ops
 - Verify using C test program

Do not put generic tasks like “coding”

- Must include includes activities regarding: design, implementation, verification and debug on FPGA
- At least one **core functionality/module of the minimum system** should be implemented and tested by Apr 12.
- Ensure minimum system (commitment) is met by **Apr 19**. This will leave some room to handle last-minute surprises.
- **Ensure parallel work distribution across team members.** Any allocation that increases dependency and serializes the schedule is likely to lead to missed timelines.
- Instructions to fill the Gantt chart:
 - First, list down activities (at least 15, more is preferable)
 - Assign dates (mark corresponding cells in green)
 - Fill the owner for each activity

[illegible]

[illegible]

[illegible]

[illegible]

7. Risks & Challenges (≤ 5 bullets, 1 line each)

Examples:

- Handling overflows
- Multi-cycle stall integration
- Signed vs unsigned errors
- FPGA timing mismatch

1. sub-cycle race conditions where the same data is accessed by multiple cores
2. ensuring integrity of inter-core atomic locks managed
3. maintaining cache coherency across multiple cores during parallel execution
4. Refactoring the baseline 3-stage pipeline into a 5-stage architecture introduces more structural and data hazards.
5. Integrating the RV32M multi-cycle sequential multiplier and 32 cycle divider increases complexity hence requiring an increase in clock cycle time period

8. Response to TA feedback (for the project abstract)

- Provide response to each input/point individually. Please see: bit.ly/cs224-ta-feedback
- Ensure you have added the TA already to the GIT repo

1. The 5-stage pipeline architecture has already been implemented. We split the modules among us and then worked on the main module i.e pipeline.v together
 2. Sure, the next reports will contain a further detailed distribution of work and the timeline for the same.
-